Project 7: Aerial Firefighting (100 points)

Please use a new page for each part of a problem. That will help your GSI and grader!

Motivation

Aerial firefighting is a critical tool in modern wildfire suppression, particularly as climate change drives longer fire seasons and larger burn areas. Fixed-wing aircraft and air tankers can deliver fire retardant over steep, inaccessible terrain far faster than ground crews, but their effectiveness depends heavily on *when*, *where*, and *how* the retardant is released. Poor drop geometry—wrong altitude, speed, or heading—can scatter retardant outside the fire perimeter entirely. Optimizing these parameters manually is infeasible given the high-dimensional design space and the complex, coupled physics involved. This project uses physics-based simulation paired with a genetic algorithm to systematically search for drop configurations that maximize retardant coverage over a target fire zone.

Governing Equations

The simulation couples aircraft path kinematics with Lagrangian particle dynamics. The aircraft follows a parametric sine-wave trajectory $\boldsymbol{r}(u) = (u + x_0,\ y_0,\ A\sin(2\pi u/\lambda) + z_0)^T$, advanced each timestep by $\Delta u = V\Delta t/\|\boldsymbol{r}'(u)\|$ to enforce constant airspeed. Retardant droplets are ejected from four nozzles with velocity $\boldsymbol{v}_{\text{drop}} = \boldsymbol{v}_{\text{aircraft}} + v_e\,\hat{\boldsymbol{n}}_{\text{spray}}$, where the spray direction $\hat{\boldsymbol{n}}_{\text{spray}}$ is stochastically perturbed to model nozzle spread. Once airborne, each droplet obeys Newton's second law,

$$m\ddot{\boldsymbol{r}} = \boldsymbol{f}_{\text{drag}} + m\boldsymbol{g}, \tag{1}$$

where the drag force is $\boldsymbol{f}_{\text{drag}} = \frac{1}{2}\rho_{\text{air}}C_D(\pi r_p^2)\|\boldsymbol{v}_{\text{rel}}\|\boldsymbol{v}_{\text{rel}}$ with $\boldsymbol{v}_{\text{rel}} = (\boldsymbol{v}_{\text{wind}} + \boldsymbol{v}_{\text{up}}) - \boldsymbol{v}_{\text{drop}}$. Above the fire footprint, a thermal updraft $v_{\text{up}}(y) = v_{\text{up,max}}\exp\big(-a_{\text{decay}}\max(0, y)\big)$ opposes descent. $v_{\text{up,max}}$ and a_{decay} are the maximum velocity of the updraft (closest to the fire) and the decay rate as the air moves away from the fire, respectively. The equations of motion are integrated via Semi-Implicit Euler, and a drop is considered successful if it lands within the prescribed fire boundary. The optimization objective is to minimize the cost

$$\Pi = W_1\left(1 - \frac{N_{\text{hit}}}{N_{\text{total}}}\right) + W_2\left(1 - \frac{y_0}{y_{\text{max}}}\right) \tag{2}$$

which maximizes the fraction of particles that hit the fire region, N_{hit} while flying as high as possible, y_0. Note, starting height of the plane is constant in this project.

Modifications and Comments on Original Paper

Several modifications were made for this assignment, but you can read the full paper that inspired this project here. First, the flight path of the plane is defined by a parametric curve, not calculated by variables such as radial velocity. We feel a curve is more intuitive and can be clearly communicated to a pilot when they fly the mission. The domain of fire is now a continuous rectangle, rather than a region of discrete targets. This was for simplicity of the simulation. This led to a modified cost function, since the original maximized particles in the fire region and targets hit. Now the objective is to maximize particles in the region and the height of the aircraft. Flying higher is desirable to enhance pilot safety.

Deliverables

Fill in the answer boxes on this report.

1. **Particle physics – drag and updraft (15 points):** In flight, each active droplet obeys

$$m\,\ddot{\boldsymbol{r}} = \boldsymbol{f}_{\text{drag}} + m\,\boldsymbol{g}, \tag{3}$$

where the aerodynamic drag force is

$$\boldsymbol{f}_{\text{drag}} = \tfrac{1}{2}\,\rho_{\text{air}}\,C_D\,(\pi r^2)\,\|\boldsymbol{v}_{\text{rel}}\|\,\boldsymbol{v}_{\text{rel}}, \quad \boldsymbol{v}_{\text{rel}} = (\boldsymbol{v}_{\text{wind}} + \boldsymbol{v}_{\text{up}}) - \boldsymbol{v}_{\text{drop}}. \tag{4}$$

The fire-induced updraft velocity (applied only above the fire footprint) is

$$v_{\text{up}}(y) = 10\,\exp\big(-0.02\,\max(0,\,y)\big)\,\hat{\boldsymbol{e}}_y, \tag{5}$$

where y is the droplet height above the ground.

(i) The simulation uses Semi-Implicit Euler integration. Write out the discrete update equations for $\boldsymbol{v}$ and $\boldsymbol{r}$ at timestep $t+1$ from timestep t.

(ii) What physical phenomenon does Equation (5) model? How does the exponential decay in y reflect the behavior of a real fire plume?

(iii) Identify one limitation of the drag model in Equation (4) .

2. (a) **Design vector $\boldsymbol{\Lambda}$ and parameter bounds (5 points):** The design vector

$$\boldsymbol{\Lambda} = \left[V,\ x_0,\ y_0,\ z_0,\ r_p,\ A_s,\ t_{\text{start}},\ t_{\text{end}},\ \dot{Q},\ v_e,\ A_c,\ \lambda_c\right] \tag{6}$$

has 12 components. The parameter ranges (from `geneticalgorithm.py`) are summarized in Table 1.

Table 1: Design vector components and their admissible ranges.

Index	Symbol	Description	Lower Bound	Upper Bound
0	V	Aircraft speed	50 m/s	100 m/s
1	x_0	Initial x-position	0 m	0 m (fixed)
2	y_0	Initial y-position	50 m	250 m
3	z_0	Initial z-position	−100 m	100 m
4	r_p	Droplet radius	0.001 m	0.01 m
5	A_s	Spray amplitude	0	0.5
6	t_{start}	Release start time	0 s	$0.1\,T$
7	t_{end}	Release end time	$0.25\,T$	T
8	$\dot{Q}$	Retardant flow rate	2 m^3/s	10 m^3/s
9	v_e	Ejection speed	0 m/s	150 m/s
10	A_c	Curve amplitude	1 m	100 m
11	λ_c	Curve wavelength	500 m	2000 m

Release times are given as fractions of the total flight time $T = 60$ s.

In a sentence, why is x_0 fixed at 0 rather than included in the optimization?

2. (b) **Implement `student_simulation.py` (40 points):** Run `python write_parameters.py` in your terminal. Open `student_simulation.py`. Rename the file `simulation.py`. Follow the instructions below then run `main.ipynb` to generate figures and numerical results. The first notebook cell turns on auto-reload, so edits to `simulation.py` are picked up without restarting the kernel. When finished, confirm your implementation reproduces a cost of **0.0508** when run with the fixed design vector:

```
LAM = [69.3, 0.0, 175.1, -11.92, 0.00429, 0.183, 0.02159, 0.6921, 8.21, 4.878, 4.85, 1520.7]
```

Setup: from the `src/` folder, run `pip install -r requirements.txt`. This installs required libraries.

(i) **Step 1 – `_update_aircraft_state`:** The aircraft travels along a parametric curve at constant airspeed V. Fill in: (a) the position and tangent from the curve at the current parameter u, (b) the step size $\Delta u = V\,\Delta t\,/\,\|\boldsymbol{t}\|$ so the aircraft advances at constant speed, and (c) the new `airplane.pos`.

(ii) **Step 2 – `_compute_nozzle_positions`:** Four nozzles are arranged around the aircraft. Fill in the lateral and aft offsets for nozzles 1, 2, and 3, and the vertical drop from the sprayer base to its end. Reference the nozzle layout comments in the file.

(iii) **Step 3 – `_compute_spray_velocities`:** The spray direction is the nozzle axis perturbed by random noise. Fill in: (a) the perturbed spray direction $\hat{\boldsymbol{n}}_{\text{base}} + A_s\,\hat{\boldsymbol{n}}_{\text{rand}}$, and (b) the final droplet velocity $\boldsymbol{v}_{\text{nozzle}} + v_e\,\hat{\boldsymbol{n}}_{\text{spray}}$.

(iv) **Step 4 – `_release_droplets`:** Fill in the one-timestep position pre-integration (`sprayer_end + DT * velocities`) so each droplet starts one step ahead of the nozzle.

(v) **Step 5 – `_in_fire_zone`:** Fill in the squared distance from each particle to the circle center $(x - x_c)^2 + (z - z_c)^2$, and the comparison against R^2. The box case is already provided for reference. The default `FIRE_ZONE_SHAPE` is `'box'` ($x \in [200, 300]$ m, $z \in [-50, 50]$ m), so the circle branch only runs if you switch the shape.

(vi) **Step 6 – `_compute_updraft`:** Fill in the updraft speed for particles inside the fire zone:

$$v_{\text{up}}(y) = V_{\text{up}} \exp\!\big(-\kappa \max(0,\, y)\big),$$

where V_{up} = `UPDRAFT_VELOCITY` and κ = `UPDRAFT_DECAY`.

(vii) **Step 7 – `_compute_drag`:** Fill in the Reynolds number $\text{Re} = 2\, r_p\, \rho_a\, |\boldsymbol{v}_{\text{rel}}| \,/\, \mu$. The Chow piecewise C_D table and the drag force assembly are already provided.

(viii) **Step 8 – `_step_physics`:** Fill in three lines using Newton's second law and Semi-Implicit Euler:

$$\begin{aligned} \boldsymbol{v}_{\text{rel}} &= (\boldsymbol{v}_{\text{wind}} + \boldsymbol{v}_{\text{up}}) - \boldsymbol{v}_{\text{curr}}, \\ \boldsymbol{a} &= \boldsymbol{f}_{\text{drag}} \,/\, (\rho_{\text{ret}}\, v_p) + \boldsymbol{g}, \\ \boldsymbol{v} &\mathrel{+}= \boldsymbol{a}\, \Delta t, \\ \boldsymbol{r} &\mathrel{+}= \boldsymbol{v}\, \Delta t. \end{aligned}$$

(ix) **Step 9 – `simulate_path_with_nozzles`:** Fill in the physics mask passed to `_step_physics`: particles that are `active` and **not** yet `landed`.

(x) **Step 10 – `calculate_cost`:** Fill in the cost function below:

$$\Pi = W_1 \left(1 - \frac{N_{\text{hit}}}{N_{\text{total}}}\right) + W_2 \left(1 - \frac{y_0}{y_{\max}}\right).$$

where N_{hit} is the number of particles that land within the fire boundary, N_{total} is the total number of tracked particles in the tank (`num_particles`), so retardant that is never released counts as a miss, y_0 is the fixed height of the aircraft, and $y_{\max}$ is the maximum possible height we allow the plane to fly. To help with debugging, confirm the cost printed is 0.0508 and uncomment the animation.

2. (c) **GA convergence (10 points):** Run the genetic algorithm from the GA cell in `main.ipynb` (which calls `geneticalgorithm.py`) using your completed simulation and plot the best cost, average cost, and parents' average cost versus generation number. Report the best design vector $\boldsymbol{\Lambda}^*$ found and its corresponding cost $\Pi(\boldsymbol{\Lambda}^*)$. Include a screenshot of the animation for the optimized design. The notebook saves the optimized design as a GIF, `../animations/ga_result.gif`; the screenshot can come from that GIF or from the final frame the notebook shows under the cell.

3. **Particle budget and flow rate (5 points):** The total number of particles in the simulation is

$$N_{\text{total}} = \left\lfloor \frac{f\, V_{\text{tank}}}{v_p} \right\rfloor, \qquad v_p = \tfrac{4}{3}\pi r_p^3, \tag{7}$$

and the number released per timestep is $\lfloor f\, \dot{Q}\, \Delta t / v_p \rfloor$, where $f = 5 \times 10^{-5}$ is the fraction of droplets that are tracked.

To keep particle counts computationally manageable with these small droplets, each of the smaller tracked droplets is a "super-particle" representing $1/f = 2 \times 10^4$ droplets.

Why is using realistic droplet sizes with a full-size tank problematic computationally, and what trade-off does the super-particle method introduce?

4. **Mini-Sensitivity Analysis (25 Points):** A sensitivity analysis tells us how much an outcome changes with respect to changes in its input variable(s). Mathworks offers a nice summary here. In this problem we examine how "sensitive" our cost is to the aircraft speed V (swept over 50–100 m/s, the same range the GA samples, 15 values) and the droplet ejection speed v_e (swept over 0–150 m/s, 15 values). We have already coded the sample: `main.ipynb` has two ready-made sensitivity cells. Run both.
Discuss the plot surfaces. Can you conclude a genetic algorithm or a convex method is better suited for this problem? Why or why not? What do we learn about cost function sensitivity when there are sudden changes (like a steep cliff) in the cost function with respect to a parameter?

Submission

1. Submit the PDF copy of your typed report on Gradescope before the end of day (11:59 pm PST).